SOFTWARE ENGINEERING – UNIT 5

Complete Semester Examination Notes

Syllabus: Product Metrics; Software Project Management; Effort Estimation; COCOMO; Risk Management; Project Tracking and Control.

These notes retain the explanations, definitions, classifications, formulas, worked examples, diagrams and exam-preparation guidance covered in the conversation. The COCOMO clarification is included as an expanded section.

PART 1: PRODUCT METRICS

1. A Framework for Product Metrics

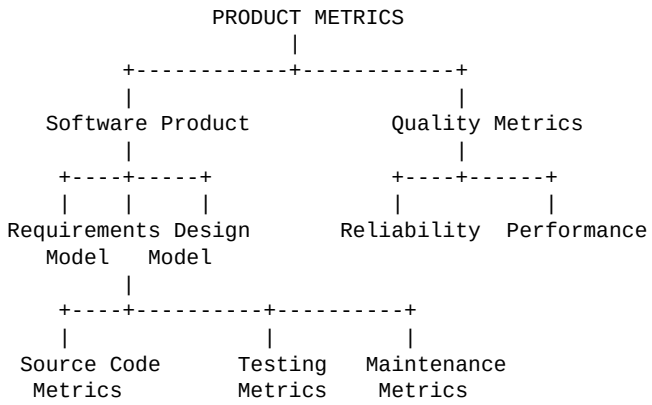
Definition

Product metrics are measurements used to evaluate the quality, performance, complexity and reliability of a software product. They help developers understand whether software meets the required standards.

Need for Product Metrics

1. To measure software quality.
2. To identify errors and defects.
3. To improve software performance.
4. To measure software complexity.
5. To estimate maintenance costs.
6. To ensure customer satisfaction.

Framework for Product Metrics



Types of Product Metrics

1. Requirements Metrics: Measure the quality and completeness of software requirements.
2. Design Metrics: Measure the quality and complexity of software design.
3. Source Code Metrics: Measure code size, complexity and readability.
4. Testing Metrics: Measure testing effectiveness and defect detection.
5. Maintenance Metrics: Measure the effort required to modify and maintain software.

Advantages

• Improves software quality.

• Helps identify problems early.

• Reduces development costs.

• Supports better decision-making.

• Improves project management.

2. Metrics for Requirements Model

Definition

Requirements metrics are measurements used to evaluate the quality, completeness, consistency and complexity of software requirements. The requirements model describes what the software should do.

Important Requirements Metrics

1. Number of Requirements

Measures the total number of functional and non-functional requirements. Example: Functional requirements = 30; Non-functional requirements = 10; Total requirements = 40.

2. Requirements Completeness

Measures whether all necessary requirements have been identified.

3. Requirements Volatility

Measures how frequently requirements change during software development.

Requirements Volatility (RV) = Number of Changed Requirements / Total Requirements

Example: Total requirements = 100; changed requirements = 20. RV = 20/100 = 0.2, therefore requirements volatility = 20%.

4. Requirements Traceability

Measures whether each requirement can be traced to its design, implementation and test cases.

5. Requirements Defect Density

Measures the number of errors found in the requirements document.

Advantages

• Identifies incomplete requirements.

• Reduces misunderstandings between customers and developers.

• Helps control requirement changes.

• Improves software quality.

3. Metrics for Design Model

Definition

Design metrics are measurements used to evaluate the quality, complexity, structure and efficiency of software design. Software design describes how the system will be developed.

Important Design Metrics

1. Architectural Metrics

Measure the overall structure of the software system. They help identify the number of components and relationships between them.

2. Cohesion

Cohesion measures how closely related the responsibilities of elements within a single module are. High cohesion is desirable; low cohesion is undesirable. Example: A login module should handle login-related operations only.

3. Coupling

Coupling measures the dependency between different modules. Low coupling is desirable; high coupling is undesirable. Example: If changing one module requires changing many other modules, coupling is high.

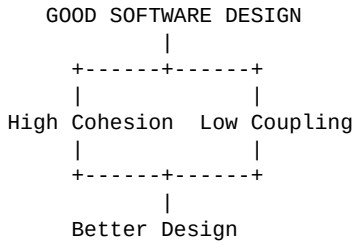
4. Complexity Metrics

Measure the complexity of the software design.

5. Interface Metrics

Measure the number and complexity of interfaces between different modules.

Important Point



Advantages

• Improves software structure.

• Makes debugging easier.

• Reduces dependencies.

• Improves maintainability.

• Supports code reusability.

4. Metrics for Source Code

Definition

Source code metrics are measurements used to evaluate the size, complexity, quality and maintainability of program code. They help developers identify complicated code and improve programming practices.

Important Source Code Metrics

1. Lines of Code (LOC)

Measures the total number of lines in a program. Example: If a program contains 500 lines of code, LOC = 500.

2. Cyclomatic Complexity

Measures the number of independent execution paths in a program.

$$V(G) = E - N + 2P$$

E = Number of edges; N = Number of nodes; P = Number of connected components (usually 1). For a single connected program: V(G) = E − N + 2. Another formula is V(G) = D + 1, where D = number of decision statements.

Example: If a program contains 4 decision statements, V(G) = 4 + 1 = 5. Therefore, the program has 5 independent paths.

3. Halstead Metrics

These metrics measure program complexity based on operators and operands.

• n1 = Number of distinct operators.

• n2 = Number of distinct operands.

• N1 = Total operators.

• N2 = Total operands.

$$\text{Program vocabulary: } n = n1 + n2$$
$$\text{Program length: } N = N1 + N2$$

4. Comment Percentage

Measures how much of the source code contains comments.

5. Code Reusability

Measures how much existing code can be reused in other applications.

Advantages

• Identifies complex code.

• Improves readability.

• Helps detect coding problems.

• Improves maintainability.

• Supports code optimization.

5. Metrics for Testing

Definition

Testing metrics are measurements used to evaluate the effectiveness, completeness and quality of software testing. They help determine whether the software has been tested properly.

Important Testing Metrics

1. Test Case Coverage

Measures the percentage of requirements covered by test cases.

$$\text{Coverage} = (\text{Requirements Tested} / \text{Total Requirements}) \times 100$$

Example: Total requirements = 100; requirements tested = 80. Coverage = (80/100) × 100 = 80%.

2. Defect Density

Measures the number of defects present in a software product relative to its size.

Defect Density = Total Defects / Software Size

3. Test Execution Rate

Test Execution Rate = (Executed Test Cases / Total Test Cases) × 100

Measures the percentage of test cases executed.

4. Defect Removal Efficiency (DRE)

Measures how effectively defects are removed before software delivery.

DRE = E / (E + D) × 100

E = Errors found before delivery; D = Errors found after delivery.

5. Test Pass Percentage

Pass Percentage = (Passed Test Cases / Executed Test Cases) × 100

Advantages

• Measures testing effectiveness.

• Helps identify remaining defects.

• Improves software reliability.

• Ensures better product quality.

6. Metrics for Maintenance

Definition

Maintenance metrics are measurements used to evaluate the effort, cost and time required to modify, correct or improve software after delivery. Software maintenance is necessary because software may require changes even after successful development.

Types of Software Maintenance

1. Corrective Maintenance: Fixing errors found after software delivery.

2. Adaptive Maintenance: Modifying software to work in a changed environment.

3. Perfective Maintenance: Improving software performance or adding new features.

4. Preventive Maintenance: Making changes to prevent future problems.

Important Maintenance Metrics

1. Mean Time to Repair (MTTR)

Measures the average time required to repair a software failure.

MTTR = Total Repair Time / Number of Repairs

2. Mean Time Between Failures (MTBF)

Measures the average time between two consecutive failures.

3. Maintenance Effort

Measures the total effort required to maintain the software.

4. Change Request Rate

Measures how frequently users request changes to the software.

5. Maintainability

Measures how easily software can be modified, corrected and improved.

Advantages

• Reduces maintenance costs.

• Improves system reliability.

• Helps plan maintenance activities.

• Identifies frequently failing modules.

• Improves software lifetime.

PART 2: SOFTWARE PROJECT MANAGEMENT

7. Definition of Software Project

Definition

A software project is a planned activity carried out to develop, design, test, implement and maintain a software application within a specified time, budget and resources.

Characteristics of a Software Project

1. Unique: Every software project has different requirements.

2. Time-bound: It must be completed within a specified period.

3. Limited Budget: Projects have financial limitations.

4. Resources: Developers, tools and hardware are required.

5. Complexity: Software projects involve technical challenges.

6. Goal-oriented: Every project has specific objectives.

Examples

• Banking management system.

• Hospital management system.

• E-commerce application.

• Online examination system.

• College management system.

Main Objectives

• Complete the project on time.

• Complete the project within budget.

• Deliver quality software.

• Satisfy customer requirements.

• Use available resources effectively.

8. Need for Software Project Management

Definition

Software Project Management (SPM) is the process of planning, organizing, scheduling, monitoring and controlling software development activities to achieve project objectives.

Why is Software Project Management Needed?

1. Time Management: Helps complete the project within the scheduled time.

2. Cost Management: Controls project expenses and avoids unnecessary spending.

3. Resource Management: Ensures proper utilization of developers, tools and equipment.

4. Risk Management: Identifies possible risks and prepares solutions.

5. Quality Management: Ensures that the developed software meets quality standards.

6. Team Coordination: Improves communication and coordination among team members.

7. Project Monitoring: Tracks project progress and identifies delays.

Advantages

• Reduces project failure.

• Improves productivity.

• Controls project cost.

• Ensures timely delivery.

• Improves customer satisfaction.

9. Software Project Management Plan (SPMP)

Definition

A Software Project Management Plan (SPMP) is a formal document that describes how a software project will be planned, executed, monitored and controlled. It acts as a roadmap for the entire software development process.

Important Components of SPMP

1. Project Overview: Contains project objectives, scope and goals.

2. Project Organization: Defines the project team structure and responsibilities.

3. Project Schedule: Specifies activities, milestones and deadlines.

4. Resource Planning: Identifies the required human resources, hardware and software.

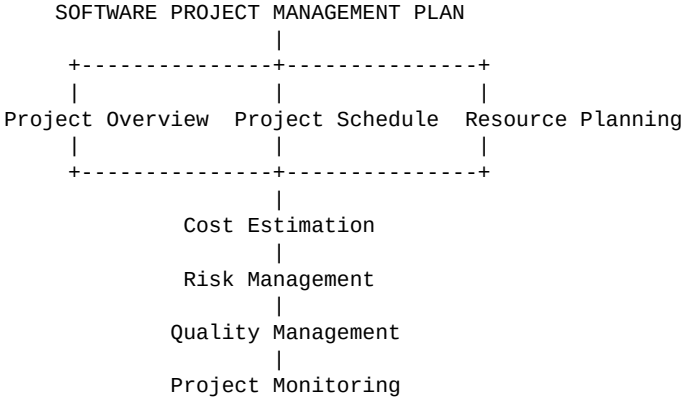
5. Cost Estimation: Estimates the total project development cost.

6. Risk Management: Identifies possible risks and their solutions.

7. Quality Management: Defines quality standards and testing procedures.

8. Monitoring and Control: Explains how project progress will be tracked.

Diagram



Advantages

• Provides clear project direction.

• Defines responsibilities.

• Helps manage time and cost.

• Reduces project risks.

• Improves coordination.

10. Effort Estimation

Definition

Effort estimation is the process of predicting the amount of work required to develop a software project. It is generally measured in Person-Months (PM). One person-month means the work completed by one person in one month.

Need for Effort Estimation

1. To calculate project development cost.

2. To determine the number of developers required.

3. To estimate project completion time.

4. To allocate resources properly.

5. To prepare project schedules.

6. To avoid project delays.

Software Effort Estimation Techniques

1. Expert Judgment: Experienced software developers estimate the effort based on their knowledge and previous experience.

2. Estimation by Analogy: Estimates the effort by comparing the current project with similar completed projects.

3. Function Point Analysis: Estimates software size based on functionality provided to users.

4. LOC-Based Estimation: Estimates effort based on the expected number of lines of code.

5. COCOMO Model: Uses mathematical equations to estimate software development effort and schedule.

Importance

• Helps in budget preparation.

• Supports resource allocation.

• Improves project planning.

• Reduces estimation errors.

11. COCOMO Model – Case Study on Effort Estimation

COCOMO stands for Constructive Cost Model. It was developed by Barry Boehm in 1981. It is a software cost estimation model used to estimate the effort, development time and manpower required for a software project. COCOMO uses software size measured in KLOC (Kilo Lines of Code).

1 KLOC = 1000 Lines of Code

Types of COCOMO Models

1. Basic COCOMO.
2. Intermediate COCOMO.
3. Detailed COCOMO.

11.1 Basic COCOMO

Basic COCOMO estimates software development effort and time based on the size of the software.

Effort: E = a(KLOC)^b

E = Effort in Person-Months; KLOC = Thousand Lines of Code; a and b = constants depending on project type.

Development Time: T = 2.5(E)^d

T = Development time in months; E = Estimated effort; d = constant depending on project type.

Average Staff Required: Staff = E / T

Three Modes of COCOMO

Project Mode	Description	Example
Organic	Small and simple projects developed by experienced teams	Payroll system
Semi-Detached	Medium-sized projects with moderate complexity	Banking system
Embedded	Complex projects with strict hardware or software constraints	Aircraft control system

Basic COCOMO Constants

Project Mode	a	b	d
Organic	2.4	1.05	0.38
Semi-Detached	3.0	1.12	0.35
Embedded	3.6	1.20	0.32

11.2 Basic COCOMO Numerical Problem

Question: A software project has an estimated size of 32 KLOC. Calculate the effort, development time and average number of staff required using Basic COCOMO. Assume the project is Organic.

Given Data

KLOC = 32; Project Mode = Organic; a = 2.4; b = 1.05; d = 0.38

Step 1: Calculate Effort

E = a(KLOC)^b
E = 2.4(32)^1.05
E = 2.4 × 38.05
E = 91.33 Person-Months

Step 2: Calculate Development Time

$$T = 2.5(E)^d$$
$$T = 2.5(91.33)^0.38$$
$$T = 13.90 \text{ Months}$$

Step 3: Calculate Average Staff Required

$$\text{Staff} = E / T = 91.33 / 13.90 = 6.57$$

Approximately 7 people are required.

Parameter	Result
Project Size	32 KLOC
Effort	91.33 Person-Months
Development Time	13.90 Months
Average Staff	Approximately 7

Conclusion: The estimated project requires approximately 91.33 person-months of effort, 13.90 months of development time and an average of 7 developers.

11.3 Intermediate COCOMO

Intermediate COCOMO considers two things: (1) software size (KLOC), and (2) cost drivers (factors affecting development effort).

$$\text{Effort: } E = a(KLOC)^b \times EAF$$

$$\text{Development Time: } T = 2.5(E)^d$$

$$\text{Average Staff: } P = E / T$$

EAF means Effort Adjustment Factor.

How to Calculate EAF?

EAF is calculated by multiplying the effort multipliers of different cost drivers.

$$EAF = EM_1 \times EM_2 \times EM_3 \times \ldots \times EM_n$$

There are 15 cost drivers in the original Intermediate COCOMO model. They are classified into four categories:

Category	Cost drivers
Product attributes	Required reliability, database size, product complexity
Hardware attributes	Execution time, storage constraints, virtual machine volatility, computer turnaround time
Personnel attributes	Analyst capability, programmer capability, application experience, virtual machine experience, programming language experience
Project attributes	Modern programming practices, software tools, required development schedule

Each cost driver has an effort multiplier (EM), depending on its rating, such as Very Low, Low, Nominal, High or Very High.

EAF Calculation – Numerical Example

Suppose a question gives the following effort multipliers:

Cost driver	Rating	Multiplier
Software reliability	High	1.15
Database size	High	1.08
Product complexity	Nominal	1.00
Programmer capability	High	0.86
Software tools	Low	1.10

Step 1: Multiply all the effort multipliers.

$$\text{EAF} = 1.15 \times 1.08 \times 1.00 \times 0.86 \times 1.10$$
$$\text{EAF} \approx 1.177$$

Step 2: Substitute EAF into Intermediate COCOMO.

Suppose: Project mode = Organic; KLOC = 10; EAF = 1.177. Intermediate Organic constants: a = 3.2, b = 1.05.

$$E = 3.2(10)^{1.05} \times 1.177$$
$$E \approx 42.26 \text{ Person-Months}$$

Important: In an examination numerical, the question may provide the effort multipliers directly or provide their ratings and ask you to refer to the COCOMO multiplier table. Multiply the applicable multipliers to obtain EAF. A nominal rating normally has a multiplier of 1.00.

11.4 Detailed COCOMO

Detailed COCOMO is an advanced version of Intermediate COCOMO. It considers the effect of cost drivers separately at each phase of the software development life cycle.

Formula

Detailed COCOMO uses the Intermediate COCOMO effort equation with phase-sensitive effort multipliers.

$$\text{General form: } E = a(KLOC)^b \times EAF$$

$$\text{For individual phases: } E_i = E_{nominal,i} \times EAF_i$$

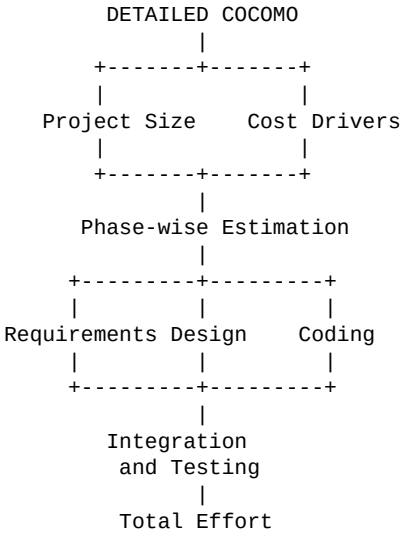
E_i = Effort for a particular phase; EAF_i = Phase-specific effort adjustment factor; E nominal,i = Nominal effort allocated to that phase.

$$\text{Overall Project Effort: } E_total = \Sigma E_i$$

Four Phases of Detailed COCOMO

1. Requirements Planning and Product Design.
2. Detailed Design.
3. Code and Unit Testing.
4. Integration and Testing.

Diagram



Advantages

1. Provides more accurate estimation.
2. Estimates effort for each development phase.
3. Considers phase-specific cost drivers.
4. Helps allocate resources effectively.
5. Useful for large and complex projects.

11.5 Difference Between Three COCOMO Models

Feature	Basic	Intermediate	Detailed
Software size	Yes	Yes	Yes
Cost drivers	No	15	15, phase-sensitive
EAF	No	Yes	Yes, phase-wise
Estimation	Overall effort	Overall effort	Phase-wise effort
Complexity	Simple	Moderate	High
Accuracy	Lower	Better	More detailed

Important correction: The Intermediate COCOMO coefficients are different from Basic COCOMO.

Project mode	Basic a	Intermediate a	b (both)	d (both)
Organic	2.4	3.2	1.05	0.38
Semi-detached	3.0	3.0	1.12	0.35
Embedded	3.6	2.8	1.20	0.32

Schedule equation for all three models: T = 2.5(E)^d. For Detailed COCOMO, effort is estimated phase-wise; schedule planning also uses the phase distribution.

Exam preparation: Practise three numerical patterns: (1) Basic COCOMO – given KLOC and project mode, calculate effort, time and staff; (2) Intermediate COCOMO – given KLOC, project mode and cost-driver ratings, calculate EAF, effort, time and staff; (3) Detailed COCOMO – given phase-wise effort distribution and multipliers, calculate effort for each phase and total effort.

Remember: EAF is the product of the effort multipliers obtained from the cost-driver table.

12. Risk Management Plan

Definition

Risk management is the process of identifying, analyzing, prioritizing and controlling possible problems that may affect a software project. A risk is an uncertain event that may negatively affect project cost, time, quality or performance.

Types of Software Risks

1. Project Risks

These risks affect the project schedule, budget and resources. Example: Developers leaving the project.

2. Technical Risks

These risks affect the technical quality and performance of software. Example: Technology failure or integration problems.

3. Business Risks

These risks affect the business success of the software. Example: The product may not attract customers.

Steps in Risk Management

1. Risk Identification: Identify possible risks that may occur during development.

2. Risk Analysis: Analyze the probability and impact of each risk.

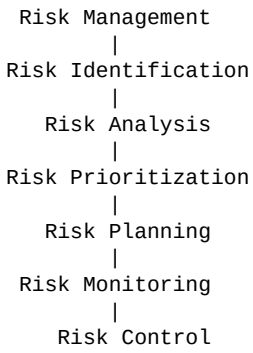
3. Risk Prioritization: Classify risks based on their severity and importance.

4. Risk Planning: Prepare strategies to avoid or reduce risks.

5. Risk Monitoring: Continuously observe risks throughout the project.

6. Risk Control: Take corrective actions when risks occur.

Risk Management Diagram



Risk Management Strategies

• Risk Avoidance: Change the project plan to eliminate a risk.

• Risk Reduction: Reduce the probability or impact of a risk.

• Risk Transfer: Transfer responsibility for a risk to another party.

• Risk Acceptance: Accept the risk and prepare to handle its consequences.

Example

Risk: A key developer may leave the project. Solution: Maintain proper documentation; share knowledge among team members; train backup developers.

Advantages

• Reduces project failure.

• Improves project stability.

• Controls unexpected costs.

• Helps complete projects on time.

13. Project Tracking and Control

Definition

Project tracking and control is the process of continuously monitoring project progress and comparing actual performance with the planned schedule, budget and objectives. It helps project managers identify delays and take corrective actions.

Project Tracking

Project tracking means observing and measuring the progress of project activities.

• Monitoring completed tasks.

• Checking project milestones.

• Measuring actual effort.

• Tracking project expenses.

• Monitoring team performance.

Project Control

Project control means taking corrective actions whenever the actual project performance differs from the planned performance.

• Identifying deviations.

• Analyzing problems.

• Taking corrective actions.

• Updating project schedules.

• Allocating additional resources when necessary.

Techniques Used in Project Tracking and Control

1. Gantt Chart

A Gantt chart represents project activities against time using horizontal bars.



2. Milestone Tracking

A milestone is an important event or achievement in the project. Examples: Requirements completed; Design completed; Coding completed; Testing completed.

3. Status Reports

Regular reports are prepared to communicate project progress.

4. Earned Value Management (EVM)

EVM is a technique used to measure project performance by comparing planned work, completed work and actual cost.

• PV = Planned Value.

• EV = Earned Value.

• AC = Actual Cost.

Schedule Variance: SV = EV − PV
Cost Variance: CV = EV − AC

Interpretation: Positive SV = Ahead of schedule; Negative SV = Behind schedule; Positive CV = Under budget; Negative CV = Over budget.

Advantages

• Identifies project delays.

• Controls project expenses.

• Improves team productivity.

• Ensures timely completion.

• Helps achieve project objectives.

IMPORTANT FORMULAS – UNIT 5

Topic	Formula
Requirements Volatility	Changed Requirements / Total Requirements
Cyclomatic Complexity	V(G) = E − N + 2P
Cyclomatic Complexity (simple)	V(G) = D + 1
Test Coverage	(Requirements Tested / Total Requirements) × 100
Defect Density	Total Defects / Software Size
Test Execution Rate	(Executed Test Cases / Total Test Cases) × 100
Defect Removal Efficiency	E / (E + D) × 100
MTTR	Total Repair Time / Number of Repairs
COCOMO Effort	E = a(KLOC)^b
COCOMO Development Time	T = 2.5(E)^d
Average Staff	E / T
Intermediate COCOMO	E = a(KLOC)^b × EAF
EAF	Product of all applicable effort multipliers
Detailed COCOMO Phase Effort	$E_i = E_{nominal,i} \times EAF_i$
Total Detailed Effort	$E_total = \Sigma E_i$
Schedule Variance	SV = EV − PV
Cost Variance	CV = EV − AC

IMPORTANT QUESTIONS FOR SEMESTER EXAM

Part A – Product Metrics

1. Explain the framework for product metrics.
2. Explain different metrics used for the requirements model.
3. Explain different metrics used for the design model.
4. Explain source code metrics with suitable examples.
5. Explain various testing metrics.
6. Explain maintenance metrics and their importance.

Part B – Software Project Management

1. Define software project management and explain its need.
2. Explain the Software Project Management Plan (SPMP) and its components.
3. What is effort estimation? Explain different effort estimation techniques.
4. Explain the COCOMO model and its three types.
5. Solve a numerical problem using Basic COCOMO.
6. Explain the Risk Management Plan and its steps.

7. Explain Project Tracking and Control with suitable diagrams.

Most Important Topics – Extra Preparation

Priority	Topic	What to Prepare
□□□	COCOMO Model	Theory + all formulas + numerical
□□□	Risk Management	Types + steps + diagram
□□□	SPMP	Definition + components + diagram
□□□	Project Tracking & Control	Gantt chart + EVM
□□□	Product Metrics Framework	Classification + diagram
□□	Source Code Metrics	LOC + Cyclomatic Complexity
□□	Testing Metrics	Coverage + DRE
□□	Requirements Metrics	Volatility + Traceability
□□	Design Metrics	Cohesion + Coupling
□□	Maintenance Metrics	Types + MTTR + MTBF

FINAL PREPARATION ADVICE

1. Theory: Learn definitions, classifications, advantages and important points for every topic.

2. Diagrams: Practise the Product Metrics Framework, SPMP, Risk Management Process and Gantt Chart.

3. Numericals: Give special attention to COCOMO. Understand how to calculate effort, development time and average staff instead of simply memorizing the formulas.

Do not skip any topic in Product Metrics just because COCOMO is important. The five types of product metrics are individually examinable as 6-mark questions.